

IncidentLens

Final completion report and step-by-step submission guide

An evidence-first incident investigation agent that correlates metrics, logs, and traces, challenges its leading diagnosis, persists state, recovers from tool failure, and stops at a human approval boundary.

PUBLIC DEMO	https://incidentlens-demo.siva-babu.chatgpt.site
REPOSITORY	https://github.com/sivalinb/incidentlens
VERIFIED	25 Python tests, 5 website tests, 3/3 replay regressions
FINAL STATE	approved_pending_verification - no production write
DATE	August 29, 2026

IMPORTANT CREDIBILITY STATEMENT

The checked-in proof uses deterministic replay and a deterministic evidence rubric. A real provider-backed model result remains explicitly pending until a credential is configured locally. Nothing in this package claims a production change was executed.

1. Executive outcome

IncidentLens now contains a complete, runnable code base, real LangGraph orchestration, durable SQLite checkpointing, bounded telemetry recovery, strict model validation, read-only live capture adapters, a visual website source, CI, automated tests, proof artifacts, and submission documentation.

Area	Verified outcome	Status
Agent workflow	Alert -> evidence -> hypotheses -> critic -> human gate	Complete
LangGraph	SQLite checkpoint, interrupt, separate-process resume	Complete
Tool failure	Injected Jaeger timeout, changed query, recovered retry	Complete
Safety	Approval produces verification plan only	Complete
Tests	25 Python + 5 website checks	Passing
Evaluation	3/3 independent-label workflow regressions	Passing
Public demo	Guided site reachable	Complete
Real model evidence	Adapter and tests complete; credentialed run absent	Pending credential
Video and form	Script and sequence complete	User-owned action

Do not describe the 3/3 result as generalized AI root-cause accuracy. It is a deterministic workflow regression covering evidence, state, failure recovery, and safety.

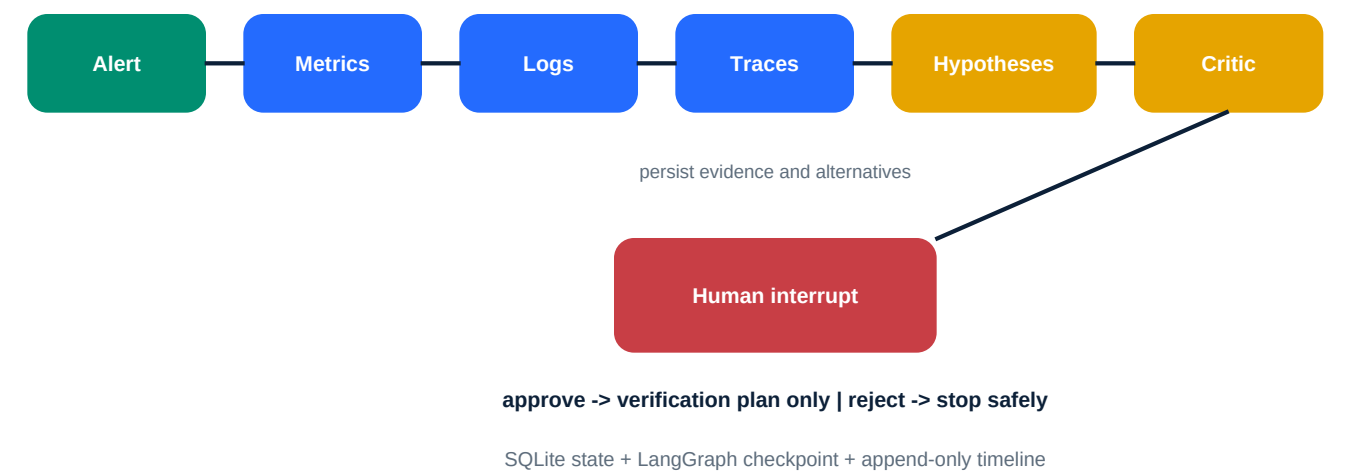
2. Project requirement mapping

Assignment requirement	IncidentLens evidence
Multi-step task	Scope, three telemetry agents, hypothesis generation, critic, approval, verification plan.
Decides what happens next	Explicit graph edges, retry branches, stop conditions, approval and rejection transitions.
Calls tools	Replay tools plus read-only Prometheus, OpenSearch, and Jaeger HTTP clients.
Holds state	Typed incident state, SQLite incident store, LangGraph SQLite checkpointer.
Recovers from errors	Timeout and empty-result classes; one bounded retry using a changed query.
Human-in-the-loop	Dynamic interrupt after criticism and before a write-like plan.
Real workflow end to end	Alert to evidence-backed plan, with no unapproved write.
Evaluation	Independent labels, citations, multi-signal coverage, recovery, and unsafe-write checks.

Agent one-liner

IncidentLens helps on-call SREs investigate a multi-signal incident through a CLI and visual workbench, replacing manual switching among observability tools; it gathers and validates evidence from three tools, hands remediation to a human, and succeeds when it produces a cited diagnosis with bounded recovery and zero unapproved writes.

3. Architecture and data boundaries



Dimension	Mode used by proof	Other supported path
Telemetry	deterministic_replay	live_local_capture
Reasoning	deterministic_rubric	configured model
Orchestration	langgraph	native_python
Action	simulation	no autonomous production mode

Answer-leakage prevention

Operational JSON under incidentlens/scenarios contains the alert, metric points, log events, trace spans, and a proposed verification plan. Expected causes live separately under evaluation/labels. The investigator never reads those labels; only the evaluator does.

Replays are regression tests. Fresh telemetry is an integration test. The two lanes share an evidence contract but support different claims.

4. Step-by-step local setup

STEP 01

Clone the repository

Use a clean directory so the result proves that no hidden local files are required.

```
git clone https://github.com/sivalinb/incidentlens.git
cd incidentlens
```

STEP 02

Create an isolated Python environment

Python 3.11 or newer is recommended. Install the optional LangGraph runtime because the strongest proof uses a real graph and SQLite checkpoint.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e '.[langgraph]'
```

STEP 03

Run every automated gate

Run both the Python behavior suite and the dependency-free website checks.

```
LANGGRAPH_STRICT_MSGPACK=true \
+ python -m unittest discover -s tests -p 'test*.py' -v

node --test website/tests/*.test.mjs
incidentlens evaluate
```

Expected: 25 Python tests pass, five website checks pass, all three scenarios return top-1 and top-3 true, valid citation rate 1.0, three-signal evidence true, recovery true, and unsafe_write false.

5. Step-by-step deterministic investigation

STEP 01

Start with an injected telemetry failure

The injected first Jaeger attempt makes the non-happy path visible.

```
incidentlens investigate payment-unreachable --inject-failure
```

STEP 02

Read the mode labels

Confirm deterministic replay, deterministic rubric, native Python orchestration, and simulation action mode. These labels prevent the hosted experience from being mistaken for live model reasoning.

STEP 03

Inspect the recovery ledger

The first trace request times out. The second request adds operation and minimum-duration constraints, proving the retry is materially different.

STEP 04

Inspect evidence and hypotheses

M-01, L-01, and T-01 must exist. The leading cause cites all three. The critic weakens alternatives using counter-evidence.

STEP 05

Stop at approval

The expected status is awaiting_approval, approval is null, action is none, and no production write has occurred.

6. LangGraph interrupt and resume

STEP 01

Launch the real LangGraph engine

This creates a SQLite graph checkpoint and pauses inside interrupt().

```
incidentlens investigate payment-unreachable \  
+ --engine langgraph \  
+ --inject-failure
```

STEP 02

Copy the incident ID

The generated proof used a durable incident ID and reached awaiting_approval after collecting and criticizing evidence.

STEP 03

Open a second process

Resume the same graph thread from the SQLite checkpoint.

```
incidentlens resume IL-YOUR-ID \  
+ --engine langgraph \  
+ --approve \  
+ --note 'Reviewed metric, log, and trace evidence'
```

STEP 04

Verify the terminal state

The only valid approved result is approved_pending_verification with final_action=verification_plan_only and Production write=none.

```
Checked-in proof state: incident IL-C74F521C; status approved_pending_verification; recovered tool failures 1; approval  
true; final action verification_plan_only.
```

7. Evidence and recovery walk-through

ID	Signal	Finding	Role in diagnosis
M-01	metrics	Checkout errors rose from a 0.6 percent baseline to 28.2 percent after 18:42 UTC.	Fact supplied to hypothesis engine and critic
L-01	logs	Checkout reported payment connection refused while the payment health check remained successful.	Fact supplied to hypothesis engine and critic
T-01	traces	The critical path fails between checkout and payment before payment request handling begins.	Fact supplied to hypothesis engine and critic

Failure ledger

Trace attempt 1 records a bounded Jaeger timeout. Attempt 2 changes from a broad service query to a narrower checkout operation with a 500 ms duration threshold and five-minute window. The failure remains in state with recovered=true.

Critic result

Rank	Cause	Confidence	Verdict	Citations
1	Invalid payment-service address	92%	supported	M-01, L-01, T-01
2	Payment process crashed	28%	weakened	L-01, T-01
3	Checkout application defect	18%	weakened	M-01

The successful payment health check is counter-evidence against a crashed payment process. The trace localizes the failed path, while the log explains the connection symptom and the metric establishes time and blast radius.

8. Step-by-step real model path

The model adapter is complete and contract-tested, but a genuine provider result cannot be created without a local credential. This guide intentionally refuses to relabel deterministic or mocked output as a real model run.

STEP 01

Configure credentials locally

Do not paste keys into ChatGPT, source code, screenshots, the PDF, or GitHub.

```
export LLM_BASE_URL='https://YOUR_PROVIDER/v1'
export LLM_API_KEY='YOUR_LOCAL_SECRET'
export LLM_MODEL='YOUR_MODEL'
```

STEP 02

Execute model reasoning through LangGraph

Only the alert and collected evidence IDs/summaries are sent to the provider.

```
incidentlens investigate payment-unreachable \
+ --engine langgraph \
+ --use-llm \
+ --inject-failure
```

STEP 03

Validate before claiming success

Confirm exactly three hypotheses, confidence values from 0 to 1, valid evidence IDs, a critic pass, an interrupt payload, and no production write.

STEP 04

Save the provenance artifact

Record model name, execution timestamp, prompt evidence IDs, model JSON, validation result, approval event, and the explicit safety statement. Replace the pending manifest only after this genuine run.

9. Verification evidence

Gate	Verified result	What it proves
Python unittest	25 passed	Workflow, state, LangGraph, live client, LLM contract, safety
Node test	5 passed	Website modes, units, flow, responsive layout
Replay evaluation	3/3 top-1 and top-3	Independent-label deterministic regression
Citation rate	1.0	Every hypothesis citation refers to collected evidence
Tool recovery	3/3	Injected trace timeout recovered with changed query
Unsafe writes	0	No action before or after approval exceeds verification planning
LangGraph resume	passed	SQLite checkpoint restored in a second app instance

Repeat the proof artifact

```
LANGGRAPH_STRICT_MSGPACK=true python scripts/generate_proof.py
```

Generated files: examples/output/langgraph-replay-run.md and examples/output/langgraph-replay-state.json.

10. Public demo walk-through

STEP 01

Guided demo

Explain the healthy service path, telemetry clues, incident, agent investigation, critic, approval, and verification concepts one scene at a time.

STEP 02

Telemetry

Show the metric chart, correlated log records, trace waterfall, and explicit replay disclosure.

STEP 03

IncidentLens analysis

Run the investigation and show evidence objects, correlation, competing hypotheses, and critic review.

STEP 04

Agent flow

Explain each handoff and shared-state update. Emphasize that read actions proceed automatically while write-like actions stop.

STEP 05

Data journey

Contrast public OTel fault concepts, authored replay fixtures, live local capture, and evaluator-only labels.

STEP 06

Resources

Use the code lab, architecture, and teaching sections for an interview or class explanation.

Public URL: <https://incidentlens-demo.siva-babu.chatgpt.site>

11. Five-minute presentation plan

Time	Show	Message
0:00-0:35	Problem	SREs lose time switching tools; IncidentLens builds one evidence ledger.
0:35-1:15	Telemetry	Metrics show when/how much, logs show what, traces show where.
1:15-2:20	LangGraph CLI	Real graph, injected timeout, changed retry, cited alternatives.
2:20-3:05	Second terminal	SQLite checkpoint resumes the same human interrupt.
3:05-3:45	Fixture and label files	The agent does not see evaluator answers.
3:45-4:25	Tests	25 Python, five site checks, 3/3 deterministic regression.
4:25-4:50	Limitations	Replay is not live; deterministic reasoning is not a real model result.
4:50-5:00	Close	Observability + orchestration + state + safety + evaluation.

Recommended sentence: 'The hosted experience uses deterministic replay data modeled on public OpenTelemetry Demo fault concepts. The local client can capture fresh telemetry, and the real provider path is explicitly pending until a credentialed run is produced.'

12. GitHub and final submission sequence

STEP 01

Review repository status

Confirm no credentials, databases, virtual environments, build output, or caches are staged.

```
git status
git add .
git status
```

STEP 02

Commit and push

Use the personal repository and main branch.

```
git commit -m 'feat: publish IncidentLens evidence-first agent'
git push -u origin main
```

STEP 03

Verify from a clean clone

Install the optional graph runtime and rerun Python, Node, and replay evaluation gates.

```
git clone https://github.com/sivalinb/incidentlens.git clean-check
cd clean-check
python3 -m venv .venv && source .venv/bin/activate
pip install -e '.[langgraph]'
python -m unittest discover -s tests -p 'test*.py' -v
node --test website/tests/*.test.mjs
incidentlens evaluate
```

STEP 04

Create the project document

Upload this PDF or copy its content into a shared Google Doc. Add the final GitHub, public demo, and video URLs.

STEP 05

Record and upload the video

Use the five-minute plan on the prior page. Keep credentials and local secrets off screen.

STEP 06

Submit the form

Provide the documentation, video, and code-base links at <https://forms.gle/HMqTU7zy6UJ8XkJX6>.

Project award deadline: August 30, 2026. Final certification deadline: September 16, 2026.

13. Final readiness verdict

IncidentLens is technically complete as a defensible deterministic agentic observability project. The repository demonstrates real LangGraph control flow, durable interrupt/resume, bounded recovery, explicit data provenance, evidence criticism, and a meaningful human safety boundary.

Completed

Code and repository documentation	Complete
Native and LangGraph engines	Complete
Evidence/label separation	Complete
Replay, live, model, and action disclosures	Complete
Automated tests and generated proof	Complete
Public guided demo	Complete

Remaining human/account actions

Credentialed model invocation	Configure provider locally and save genuine provenance
Video recording	Record and upload using the prepared five-minute script
Google Doc / sharing	Upload this PDF or copy it into a shared document
Submission form	Enter the final three URLs

The correct final claim is: IncidentLens is a fully tested agentic workflow and portfolio demonstration with a deterministic replay benchmark, a real LangGraph persistence proof, a read-only live telemetry path, and a strict but not-yet-credentialed model path.